

S&P 500 Direction Prediction

Machine Learning Pipeline, Backtesting, and Performance Analysis

1. Project Overview

1.1 Objective

The objective of this project is to develop a **binary classification machine learning model** capable of predicting whether the **S&P 500 Index (^GSPC)** will close higher on the following trading day compared with the current day's closing price.

The prediction problem is formulated as:

Will tomorrow's S&P 500 closing price be higher than today's closing price?

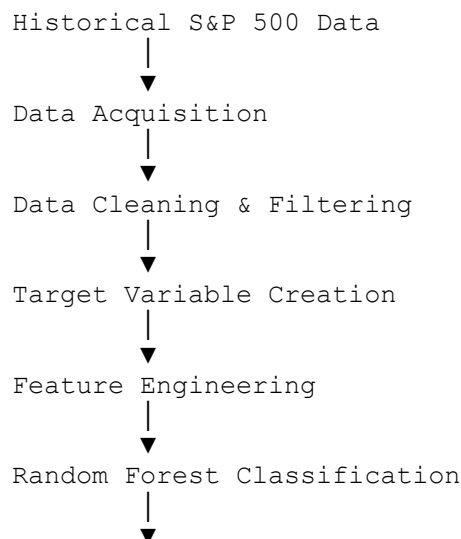
The model produces one of two classifications:

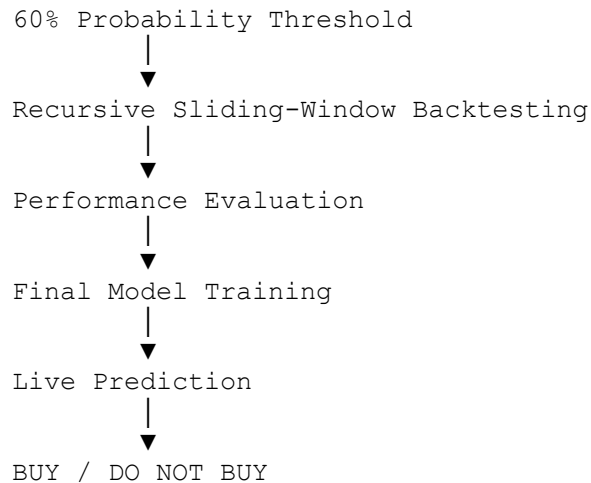
Class	Meaning
1	S&P 500 is expected to close higher tomorrow
0	S&P 500 is not expected to close higher tomorrow

Although the model ultimately produces a `BUY` or `DO NOT BUY` action, the underlying machine learning problem is a **binary classification task**, not direct price prediction.

1.2 Project Pipeline

The complete pipeline consists of the following stages:





The methodology is designed to avoid **look-ahead bias** during evaluation by ensuring that predictions are generated using information that would have been available at the time of each prediction.

2. Dataset and Data Preprocessing

2.1 Data Source

Historical S&P 500 data is obtained using the Python `yfinance` library.

The ticker used for the S&P 500 Index is:

`^GSPC`

The dataset contains daily market information including fields such as:

- Open
- High
- Low
- Close
- Volume
- Dividends
- Stock Splits

The maximum available historical data is initially downloaded, after which a temporal filter is applied.

2.2 Temporal Filtering

Only data from **January 1, 1990 onward** is retained.

```
Dataset:
Maximum available historical data
    ↓
Filter dates >= 1990-01-01
    ↓
Modern S&P 500 dataset
```

Why start from 1990?

The project assumes that financial markets from the modern period are more representative of the environment in which the model will be used.

Market structure has changed substantially over the decades due to factors such as:

- Electronic trading
- Changes in market regulation
- Changes in monetary policy
- Increased automation
- Changes in market participants
- Technological development

Therefore, older historical observations may not represent modern market behavior as effectively.

2.3 Data Cleaning

The `Dividends` and `Stock Splits` columns are removed.

```
data = data.drop(columns=["Dividends", "Stock Splits"])
```

The reasoning is that `^GSPC` represents an **index rather than an individual stock**, making these fields unnecessary for the prediction task.

Removing irrelevant columns simplifies the dataset and prevents the model from considering information that provides no useful predictive value.

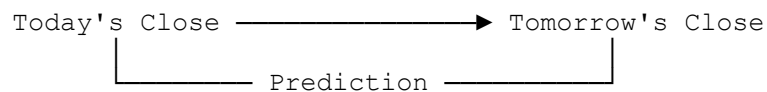
3. Target Variable Creation

The target is constructed from the next trading day's closing price.

3.1 Tomorrow's Closing Price

A new column named `Tomorrow` is created by shifting the `Close` column backward by one trading day.

Conceptually:



The operation is equivalent to:

```
data["Tomorrow"] = data["Close"].shift(-1)
```

For each trading day, `Tomorrow` therefore represents the following trading day's closing price.

3.2 Binary Target

The classification target is then created by comparing today's closing price with tomorrow's closing price.

```
data["Target"] = (data["Tomorrow"] > data["Close"]).astype(int)
```

The resulting definition is:

```
Tomorrow > Today
├── Yes → Target = 1
└── No  → Target = 0
```

Therefore:

- `Target = 1` → Up day
- `Target = 0` → Down or flat day

This transforms the original financial time series into a supervised **binary classification problem**.

4. Feature Engineering

Financial markets contain patterns across different time horizons. Instead of relying solely on individual daily prices, the model uses historical rolling information to create predictive features.

Five historical horizons are used:

2 days
5 days
60 days
250 days
1000 days

These horizons allow the model to examine different types of market behavior:

Horizon	General purpose
2 trading days	Very short-term movement
5 trading days	Short-term movement
60 trading days	Medium-term behavior
250 trading days	Approximately one trading year
1000 trading days	Long-term market behavior

The source identifies these rolling horizons as the basis for the model's engineered predictors.

4.1 Close Price Ratio

For each historical horizon, a `Close_Ratio_X` feature is calculated.

For example:

```
Close_Ratio_2  
Close_Ratio_5  
Close_Ratio_60  
Close_Ratio_250  
Close_Ratio_1000
```

The purpose of the ratio is to describe the current closing price relative to the historical rolling average.

Conceptually:

$$\frac{\text{Current Close}}{\text{Rolling Average}}$$

This gives the model information about whether the current market level is relatively high or low compared with its recent historical behavior.

4.2 Trend Accumulation

A second feature family is created using the `Trend_X` naming convention:

```
Trend_2
```

Trend_5
Trend_60
Trend_250
Trend_1000

These indicators measure the accumulation of upward movement over each historical horizon.

Conceptually, the feature answers:

"How often has the market been moving upward during this historical window?"

The combination of `Close_Ratio_X` and `Trend_X` allows the model to consider both:

1. **Current price position relative to historical levels**
2. **Accumulated upward/downward market behavior**

The source specifically identifies these as the two primary rolling indicators used by the model.

5. Machine Learning Model

5.1 Model Selection

The selected algorithm is the:

Random Forest Classifier

A Random Forest consists of multiple decision trees whose predictions are combined to produce the final classification.

The model was selected because financial datasets can contain:

- Noisy relationships
- Non-linear interactions
- Complex feature relationships
- High levels of randomness

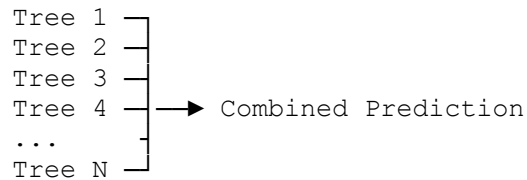
Random Forests are particularly suitable for this type of problem because they can capture non-linear relationships without requiring feature scaling.

5.2 Why Random Forest?

Resistance to Overfitting

Random Forest uses multiple decision trees and combines their predictions through ensemble learning.

Instead of relying on one tree:



This reduces the dependence on any individual decision tree.

Non-Linear Relationships

Financial indicators can interact in complicated ways.

For example:



Random Forest can model these non-linear interactions more naturally than a simple linear classifier.

No Feature Scaling Required

Unlike algorithms such as logistic regression, SVMs, or neural networks, Random Forest does not require conventional feature scaling for the decision trees to operate effectively.

The project therefore does not need to normalize all predictors before training.

These were the principal reasons given for selecting Random Forest.

6. Model Hyperparameters

Two important hyperparameters were tuned for the model.

6.1 Number of Trees

```
n_estimators = 200
```

The Random Forest contains **200 decision trees**.

Increasing the number of trees helps stabilize the ensemble's predictions and probability estimates while reducing variance.

6.2 Minimum Samples for Splitting

```
min_samples_split = 500
```

This is a deliberately large value.

A decision tree can otherwise create increasingly specific rules based on small subsets of the training data.

Using:

```
min_samples_split = 500
```

acts as a strong regularization mechanism.

The goal is to prevent the model from learning extremely specific short-term market patterns that may simply represent noise.

Instead, the model is encouraged to learn broader market relationships.

The selected hyperparameters are documented in the source as `n_estimators=200` and `min_samples_split=500`.

7. Custom Classification Threshold

7.1 Standard Classification

Most binary classification systems use a default probability threshold of:

50%

For example:

Probability of Up = 0.52



Prediction = 1

However, this project uses a stricter threshold.

7.2 60% Probability Threshold

The model generates class probabilities using:

```
preds = model.predict_proba(test[predictors])[:, 1]
```

The prediction is then converted using:

```
preds[preds >= 0.6] = 1  
preds[preds < 0.6] = 0
```

Therefore:

```
Probability >= 60%  
    ↓  
    BUY  
  
Probability < 60%  
    ↓  
    DO NOT BUY
```

This means the model does **not** issue a buy signal simply because the probability is greater than 50%.

It requires at least **60% predicted probability**.

8. Why Use a 60% Threshold?

The primary objective is to prioritize **precision over recall**.

In a trading context, an incorrect positive prediction can result in entering a position when the market subsequently falls.

Therefore, the project deliberately sacrifices prediction frequency in exchange for greater confidence.

50% Threshold

```
More predictions  
    ↓  
More BUY signals  
    |
```

▼
More potential false positives

60% Threshold

Higher confidence requirement

↓
Fewer BUY signals

↓
Greater selectivity

The model therefore behaves as a **highly selective classifier** rather than attempting to predict every market movement.

The source explicitly frames this as a precision-versus-recall and trading-frequency tradeoff.

9. Backtesting Methodology

Evaluating a financial prediction model requires special care.

A normal random train/test split would be inappropriate because it can allow information from the future to influence the training process.

This is known as:

Look-ahead bias

To avoid this problem, the project uses a **recursive sliding-window backtesting framework**.

9.1 Initial Training Window

The model initially trains on:

2,500 trading days

This corresponds to approximately:

10 years of trading data

The model then predicts the following period.

9.2 Step Size

The backtest uses:

```
step = 250
```

Therefore, after each prediction period, approximately one year's worth of new data is incorporated into the training set.

The process can be visualized as:

```
Training
<----- 2500 days ----->
```

```
                Prediction
                <--- 250 days --->
```

```
Training
<----- 2750 days ----->
```

```
                Prediction
                <--- 250 --->
```

```
Training
<----- 3000 days ----->
```

```
                Prediction
                <--- 250 --->
```

This creates a realistic scenario in which the model is periodically retrained as new historical data becomes available.

The documented backtesting configuration uses an initial size of 2,500 trading days and 250-day increments.

10. Backtesting Results

The model was evaluated over:

```
6,759 trading days
```

The results reveal an important distinction between **selectivity** and **actual predictive advantage**.

10.1 Model Precision

The model achieved:

53.59% Precision

This means that among the days on which the model issued a buy recommendation, approximately 53.59% were actually upward days.

10.2 Market Baseline

The S&P 500 itself went up on approximately:

53.69% of the days

Therefore:

Model precision	= 53.59%
Market baseline	= 53.69%
Difference	= -0.10 percentage points

This is a crucial result.

The model's buy-signal precision is **slightly below** the natural frequency with which the S&P 500 went up in the dataset.

The source explicitly reports these figures.

11. Trading Frequency

The model generated a buy signal on only:

362 / 6,759 days

That represents approximately:

5.3% of trading days

In other words, the model filtered out roughly **95% of trading days**.

6,759 total days	
— 362 BUY signals	
— ~6,397 days without BUY signal	

This demonstrates that the 60% threshold makes the classifier extremely selective.

However, being selective does **not automatically mean being more accurate**.

That is the key lesson from this backtest.

12. Performance Interpretation

The results can be summarized as follows:

Metric	Result
Backtested trading days	6,759
Buy signals	362
Buy frequency	~5.3%
Model precision	53.59%
Market upward baseline	53.69%
Model vs. baseline	-0.10 percentage points
Probability threshold	60%

The model successfully reduces the number of trading signals, but the resulting signals do not demonstrate a meaningful improvement over the market's natural upward bias.

Therefore, the model should **not be described as outperforming the S&P 500 based on this evaluation**.

The source explicitly concludes that raising the threshold to 60% significantly reduces trading opportunities without substantially outperforming the index's natural upward tendency.

13. What the Results Tell Us

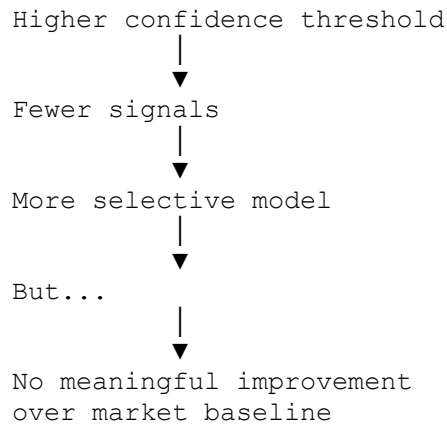
This experiment demonstrates an important property of financial machine learning:

Predicting short-term market direction is extremely difficult.

The Random Forest can identify patterns that satisfy its confidence threshold, but those patterns do not translate into a precision advantage over the market baseline in the reported backtest.

The model therefore provides an important experimental result even though the performance is not superior to the baseline.

Main observation



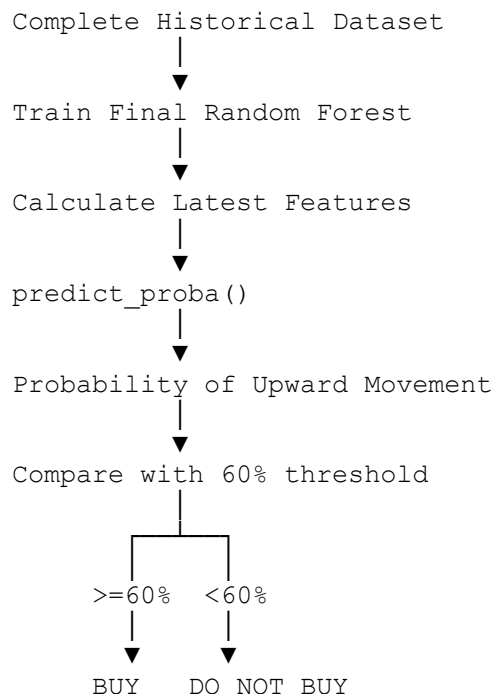
This is an important distinction between **model selectivity** and **model effectiveness**.

14. Live Prediction

After completing the backtesting process, the final model can be trained using the available historical dataset.

The final model is then applied to the most recent feature values.

The workflow is:



The final classification therefore represents the model's current assessment of whether the S&P 500 is likely to close higher on the following trading day.

15. Important Limitation

The model's output should not be interpreted as a guaranteed trading recommendation.

The reported backtest shows:

```
Model precision = 53.59%  
Market baseline = 53.69%
```

Therefore, the current implementation has **not demonstrated an advantage over the baseline**.

Additionally, precision alone does not provide a complete picture of trading performance.

A production trading system would also need to consider factors such as:

- Transaction costs
- Slippage
- Position sizing
- Risk management
- Drawdown
- Sharpe ratio
- Returns
- Maximum loss
- Portfolio exposure
- Market regimes

These measures are not provided in the supplied material, so no claims about them should be made from the current experiment.

16. Key Findings

Finding 1 — Modern historical data was selected

The dataset was restricted to data from 1990 onward to focus on more modern market behavior.

Finding 2 — Multiple time horizons were used

The model incorporates rolling information across:

```
2 → 5 → 60 → 250 → 1000 trading days
```

This gives the classifier access to short-, medium-, and long-term market information.

Finding 3 — Random Forest was selected

The model was chosen because it can capture non-linear relationships, does not require feature scaling, and uses ensemble learning to reduce variance.

Finding 4 — Strong regularization was applied

The use of:

```
min_samples_split = 500
```

helps prevent the trees from learning overly specific patterns.

Finding 5 — The threshold was increased

The standard 50% classification threshold was replaced with:

```
60%
```

to prioritize precision and reduce false-positive buy signals.

Finding 6 — The model is highly selective

Only:

```
362 / 6,759
```

backtested days generated buy signals.

Finding 7 — Selectivity did not produce an advantage

Despite being highly selective, the model achieved:

```
53.59% precision
```

versus a:

```
53.69% market baseline
```

Therefore, the model did not outperform the natural upward tendency of the S&P 500 in this experiment.

17. Potential Future Improvements

The current experiment provides a useful baseline, but there are several possible directions for improving the system.

17.1 Add External Market Features

Additional explanatory variables could include:

- Interest rates
- Inflation
- VIX
- Macroeconomic indicators
- Financial news sentiment

These features could provide information that cannot be obtained solely from historical S&P 500 prices.

The source specifically recommends incorporating macroeconomic variables, volatility measures, and sentiment analysis.

17.2 Test Other Machine Learning Models

The Random Forest should not necessarily be treated as the final model.

Potential alternatives include:

XGBoost
LightGBM
LSTM

Gradient-boosting models may capture different non-linear relationships, while recurrent neural networks such as LSTM can be evaluated for sequential market behavior.

17.3 Evaluate More Complete Trading Metrics

Future versions should evaluate not only classification precision but also actual trading performance.

Useful metrics would include:

Total Return
Annualized Return
Maximum Drawdown
Sharpe Ratio
Win Rate
Profit Factor
Number of Trades
Transaction Costs
Buy-and-Hold Comparison

This would allow the project to answer a more meaningful question:

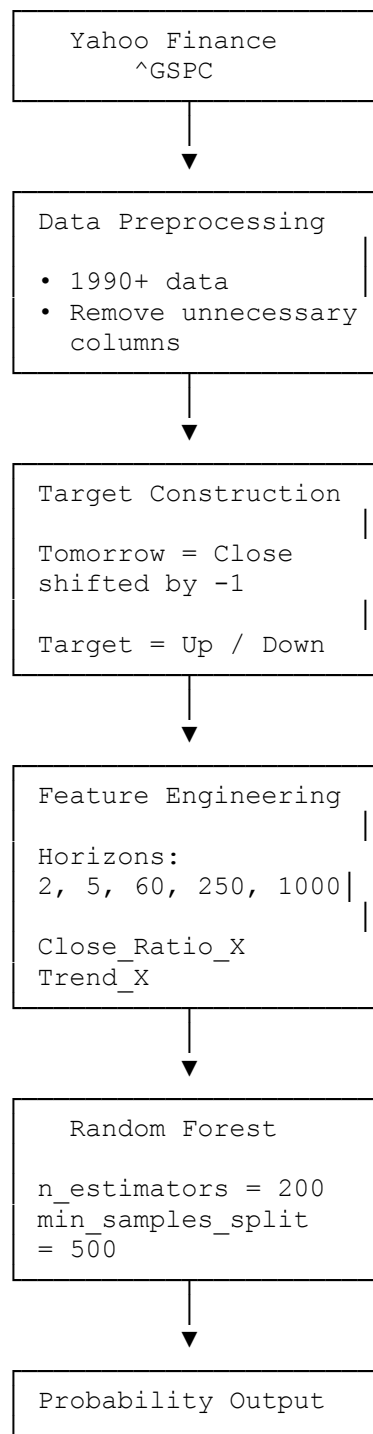
Does the model actually generate a better risk-adjusted investment strategy?

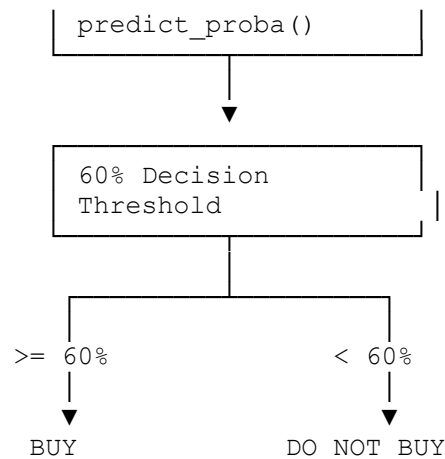
rather than simply:

Does the model correctly classify tomorrow's direction?

18. Complete Machine Learning Pipeline

The complete system can be summarized as:





19. Final Conclusion

This project implements a complete machine learning pipeline for predicting the **next-day direction of the S&P 500 Index**.

The system combines:

- Historical $\wedge_{\text{SPC}}$ data from Yahoo Finance
- Temporal filtering from 1990 onward
- Target-variable construction
- Multi-horizon rolling features
- Random Forest classification
- Strong tree regularization
- A custom 60% probability threshold
- Recursive sliding-window backtesting
- Final live prediction

The experiment demonstrates that the model can be made **highly selective**, issuing buy signals on only approximately 5.3% of backtested days.

However, the most important result is that this selectivity did **not** translate into superior predictive performance.

The model achieved:

53.59% precision

while the S&P 500 had a natural upward-day frequency of:

53.69%

Therefore, the current model should be considered an **experimental baseline rather than a proven profitable trading strategy**.

The result reinforces a fundamental challenge in financial machine learning: historical price patterns alone may not contain enough information to consistently predict short-term market direction beyond the market's natural bias.

Future work should therefore focus on richer external features, alternative models, and—most importantly—actual risk-adjusted trading performance rather than classification accuracy alone.